Trabajo práctico final: Base de datos libre

Martinez Lucia, Quiroz Ornella

Tecnicatura Superior en Desarrollo de Software - Escuela Superior de Comercio N° 43

3° - Base de Datos II – Gestión de Proyectos de Software

Ariel Nardelli, Matias Nardelli

15 de octubre, 2026

ÍNDICE

ÍNDICE **1**
INTRODUCCIÓN **2**
PRIMERA FORMA NORMAL (1FN) 2
SEGUNDA FORMA NORMAL (2FN) 3
TERCERA FORMA NORMAL (3FN) 3
TABLAS **4**
DIAGRAMA ENTIDAD-RELACIÓN **6**
SENTENCIAS DDL **11**
VISTAS **20**
CONSULTAS MÁS FRECUENTES **22**
BIBLIOGRAFÍA **23**

INTRODUCCIÓN

En el presente trabajo se demuestra el análisis y proceso de normalización de un conjunto de datos. El dataset seleccionado y utilizado en este trabajo llamado Hotel Booking contiene toda la información relacionada con las reservas de dicho hotel en una misma tabla. En ella se encontraban datos como cliente, hoteles, países, comidas, tipos de habitación, canales de distribución, entre otros.

El problema de esto es al almacenarse toda la información en una misma tabla, algunos datos se repetían varias veces, por ejemplo

Al realizar el proceso de normalización el objetivo fue reducir la redundancia de información, evitar inconsistencias y organizar los datos en tablas relacionadas entre sí mediante claves primarias y foráneas.

PRIMERA FORMA NORMAL (1FN)

Se considera que una base de datos está en su primera forma normal cuando cumple con dos requisitos, estos son:

- Todos los datos son atómicos. (Un registro se considera atómico cuando a cada información se le reserva una celda propia.)
- Todas las columnas contienen un único tipo de dato.

En este caso se analizaron los atributos de dataset para identificar los valores que podían llegar a almacenarse individualmente y evitar la repetición de información. Cada fila de la tabla original representa un registro y contiene información de distintas áreas, como el hotel, el huésped, el país de origen, el tipo de comida, el canal de distribución y el estado de la reserva.

SEGUNDA FORMA NORMAL (2FN)

Para llegar a la segunda forma normal se le agrega a los requisitos de la forma anterior el siguiente:

- Los atributos que no forman parte de ninguna clave han de depender funcionalmente de toda la clave primaria.

Entonces se verificó que los atributos de las tablas dependiera completamente de sus respectivas claves primarias. En la tabla reserva la clave primaria es ***'cod_reserva'***, que está formada por un solo atributo. Por lo tanto, los datos propios de cada reserva dependen completamente de cod_reserva.

Sin embargo, esto no significa que todos los atributos de la tabla original deban permanecer en Reserva. Durante el diseño se identificaron datos que describen otras entidades, como los hoteles, los clientes y los tipos de habitación, y se organizaron en tablas independientes relacionadas mediante claves foráneas.

TERCERA FORMA NORMAL (3FN)

Para que una tabla esté en la tercera forma normal debe de cumplir los requisitos anteriores y además:

- Los atributos no-clave no pueden depender de forma transitiva de una clave candidata.

Es decir, no se puede permitir que un atributo que no es clave dependa de otro atributo que tampoco es clave. Los datos deben depender directamente de la clave primaria de su tabla.

En nuestro caso, analizamos los atributos del dataset original , identificando los datos que se repetían y las relaciones existentes entre ellos. A partir de este análisis, observamos que algunos atributos describen entidades diferentes de la reserva, por lo que decidimos separarlos en tablas independientes, relacionadas mediante claves primarias y foráneas.

De esta manera, buscamos reducir la redundancia de información, evitar inconsistencias y facilitar la actualización de los datos.

TABLAS

NOMBRE DE LA TABLA	FUNCIÓN
Hotel	Contiene los hoteles disponibles y permite identificar en qué hotel se realiza cada reserva.
Cliente	Guarda los datos personales y de contacto de cada cliente, como nombre, apellido, teléfono, email y país.
Pais	Almacena los países asociados a los clientes, evitando repetir la descripción del país en cada registro.
Comida	Contiene los diferentes tipos de comida disponible y permite asociarlos a cada reserva.
Segmento_mercado	Almacena los diferentes segmentos de mercado a los que pertenecen las reservas.
Canal_distribucion	Identifica el canal mediante el cual se realizó cada reserva.
Tipo_habitacion	Almacena los distintos tipos de habitación. Se utiliza tanto para indicar la habitación reservada como la habitación finalmente asignada.

Tipo_cliente	Clasifica a los clientes según el tipo correspondiente.
Tipo_deposito	Contiene los diferentes tipos de depósito utilizados y permite indicar cual corresponde a cada reserva.
Estado_reserva	Contiene los diferentes estados que puede tener una reserva, permitiendo identificar la situación de cada una.
Reserva	Registra y guarda todos los datos relacionados a la contratación de habitaciones del hotel, tales como: fechas de reserva, cantidad de huéspedes, precios promedio, estacionamiento, etc.

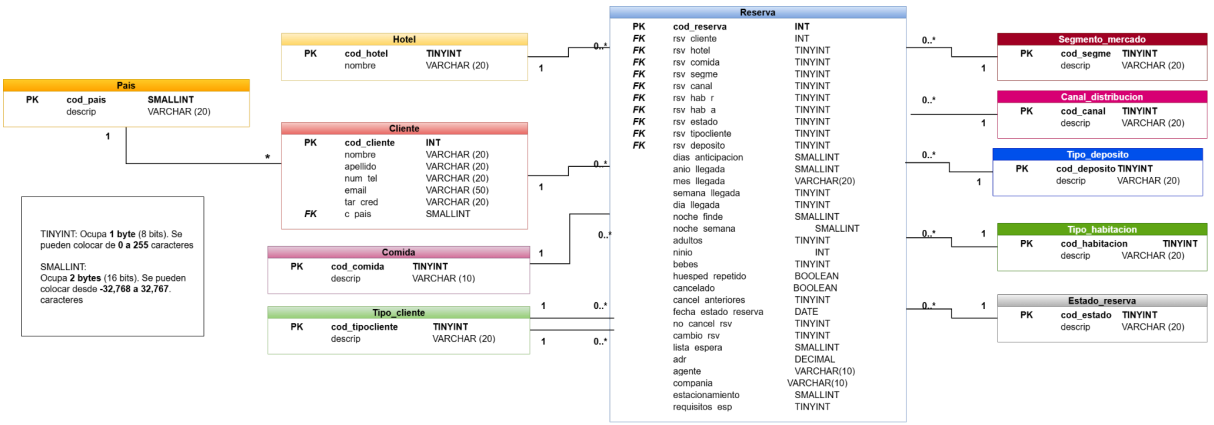
Durante el desarrollo de este proyecto se tomaron varias decisiones que facilitaron el desarrollo de la base de datos:

Agente y Compañía:

Se decidió conservar los identificadores de agente y compañía como atributos de la tabla Reserva, en lugar de crear tablas independientes.

Esta decisión se tomó teniendo en cuenta la información disponible en el dataset original, que proporciona identificadores numéricos, pero no suficientes atributos descriptivos para caracterizar estas entidades de manera independiente dentro del alcance del trabajo.

DIAGRAMA ENTIDAD-RELACIÓN



ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_hotel	TINYINT	PK	Identificador único del hotel
nombre	VARCHAR		Nombre del hotel

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_pais	SMALLINT	PK	Identificador único del país
descrip	VARCHAR		Nombre del país

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_comida	TINYINT	PK	Identificador único de comida
descrip	VARCHAR		Tipo de servicio de comida

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_tipocliente	TINYINT	PK	Identificador único del tipo de cliente

descrip	VARCHAR		tipo de cliente

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_deposito	TINYINT	PK	Identificador único del depósito
descrip	VARCHAR		tipo de depósito

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_habitacion	TINYINT	PK	Identificador único de habitación
descrip	VARCHAR		tipo de habitación

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_segme	TINYINT	PK	Identificador único del segmento de mercado
descrip	VARCHAR		segmento de mercado

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_canal	TINYINT	PK	Identificador único del canal de distribución
descrip	VARCHAR		Canal de distribución

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_estado	TINYINT	PK	Identificador único del estado de la reserva
descrip	VARCHAR		Estado de la reserva

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN

cod_cliente	INT	PK	Identificador unico del cliente
nombre	VARCHAR		Nombre del cliente
apellido	VARCHAR		Apellido del cliente
num_tel	VARCHAR		Numero telefonico del cliente
email	VARCHAR		Email del cliente
tar_cred	VARCHAR		Tarjeta de crédito del cliente
c_pais	VARCHAR	FK	País de origen

ATRIBUTO	TIPO DATO	CLAVE	DESCRIPCIÓN
cod_reserva	INT	PK	Identificador único del estado de la reserva
dias_anticipacion	SMALLINT		Estado de la reserva
anio_llegada	SMALLINT		Año de llegada al hotel
mes_llegada	VARCHAR		Mes de llegada al hotel
semana_llegada	TINYINT		Número de semana de llegada al hotel
dia_llegada	TINYINT		Día de llegada al hotel.
noche_finde	SMALLINT		Noches de fin de semana que se reservaron.
noche_semana	SMALLINT		Noches de días de semana que se reservaron.
adultos	TINYINT		Número de adultos registrados
ninios	INT		Número de niños registrados
bebes	TINYINT		Numero de bebes registrados

huesped_repetido	BOOLEAN		Cantidad de veces que un huésped se hospedó.
cancelado	BOOLEAN		
cancel_anteriores	TINYINT		Cantidad de reservas canceladas que tuvo un huésped .
fecha_estado_reserva	DATE		Fecha en la que se estableció el último estado
no_cancel_rsv	TINYINT		Cantidad de reservas no canceladas registradas.
cambio_rsv	TINYINT		Cambios en la reserva de habitaciones
dias_lista_espera	SMALLINT		Días de espera para reservas.
adr	DECIMAL		Precio promedio que paga un huésped por una habitación.
agente	VARCHAR		Intermediario que realizó la reserva.
compania	VARCHAR		Empresa que gestionó la reserva.
estacionamiento	SMALLINT		Espacio asignado para estacionamiento
requisitos_esp	TINYINT		Total de pedidos especiales que tuvo un huésped
rsv_cliente	INT	FK	Información del cliente.
rsv_hotel	TINYINT	FK	Hoteles registrados.
rsv_comida	TINYINT	FK	Tipo de servicio de comida.
rsv_segme	TINYINT	FK	Segmentación del mercado.

rsv_canal	TINYINT	FK	Canal de distribución.
rsv_hab_r	TINYINT	FK	Tipo de habitación reservada.
rsv_hab_a	TINYINT	FK	Tipo de habitación asignada.
rsv_estado	TINYINT	FK	Estado en que se encuentra la reserva.
rsv_tipocliente	TINYINT	FK	Tipo de cliente.
rsv_deposito	TINYINT	FK	Tipo de depósito.

Antes de realizar la normalización, analizamos los atributos del dataset original *Hotel Booking* para identificar qué información representaba cada uno y de qué otros atributos dependía.

Una dependencia funcional se produce cuando un atributo determina el valor de otro. Identificar estas dependencias permite reconocer qué datos pertenecen a una misma entidad y cuáles deben organizarse en tablas independientes.

Por ejemplo, en la tabla Hotel, el identificador del hotel determina sus datos descriptivos. De la misma manera, en la tabla País, el identificador del país permite identificar su nombre.

En el modelo desarrollado, la clave primaria de cada tabla permite identificar un registro de manera única, mientras que las claves foráneas establecen las relaciones entre las distintas entidades.

Este análisis fue importante para decidir cómo distribuir los atributos entre las tablas y reducir la repetición innecesaria de información.

SENTENCIAS DDL

CREAR BASE DE DATOS:

```
CREATE DATABASE reserva_hotel;
USE reserva_hotel;
```

HOTEL:

```
CREATE TABLE hotel (
    cod_hotel TINYINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(20) NOT NULL );
```

INSERT:

```
INSERT INTO hotel (nombre)
SELECT DISTINCT hotel
FROM hotel_booking_original;
```

PAÍS:

```
CREATE TABLE pais (
    cod_pais SMALLINT AUTO_INCREMENT NOT NULL PRIMARY KEY,
    descrip VARCHAR(20) NOT NULL );
```

INSERT :

```
INSERT INTO pais (descrip)
SELECT DISTINCT country
FROM hotel_booking_original;
```

TIPO CLIENTE:

```
CREATE TABLE tipo_cliente (
        cod_tipocliente TINYINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
        descrip VARCHAR(20) NOT NULL );
```

INSERT:

```
        INSERT INTO tipo_cliente (descrip)
        SELECT DISTINCT (customer_type)
        FROM hotel_booking_original;
```

COMIDA:

```
CREATE TABLE comida (
        cod_comida TINYINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
        descrip VARCHAR(10) NOT NULL );
```

INSERT:

```
        INSERT INTO comida(descrip)
        SELECT DISTINCT(meal)
        FROM (hotel_booking_original)
```

SEGMENTO MERCADO:

```
CREATE TABLE segmento_mercado (
        cod_segme TINYINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
        descrip VARCHAR(20) NOT NULL );
```

INSERT :

```
INSERT INTO segmento_mercado(descrip)
SELECT DISTINCT(market_segment)
FROM hotel_booking_original;
```

CANAL DISTRIBUCIÓN:

```
CREATE TABLE canal_distribucion (
        cod_canal TINYINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
        descrip VARCHAR(20) NOT NULL );
```

INSERT :

```
INSERT INTO canal_distribucion(descrip)
SELECT DISTINCT (distribution_channel)
FROM hotel_booking_original;
```

TIPO DEPOSITO:

```
CREATE TABLE tipo_deposito (
        cod_deposito TINYINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
        descrip VARCHAR(20) NOT NULL );
```

INSERT :

```
INSERT INTO tipo_deposito(descrip)
```

```
SELECT DISTINCT(deposit_type)
FROM hotel_booking_original;
```

TIPO HABITACIÓN:

```
CREATE TABLE tipo_habitacion (
        cod_habitacion TINYINT NOT NULL AUTO_INCREMENT PRIMARY
        KEY,
        descrip VARCHAR(20) NOT NULL );
```

INSERT :

```
INSERT INTO tipo_habitacion(descrip)
SELECT DISTINCT(reserved_room_type )
FROM hotel_booking_original;
UNION
SELECT DISTINCT(assigned_room_type )
FROM hotel_booking_original;
```

ESTADO RESERVA:

```
CREATE TABLE estado_reserva (
    cod_estado  TINYINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    descrip VARCHAR(20) NOT NULL );
```

INSERT :

```
INSERT INTO estado_reserva(descrip)
SELECT DISTINCT(reservation_status)
FROM hotel_booking_original;
```

CLIENTE:

```
CREATE TABLE cliente (
        cod_cliente INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
        nombre VARCHAR(20) NOT NULL,
        apellido VARCHAR(20) NOT NULL,
        num_tel VARCHAR (20) NOT NULL,
        email VARCHAR(50) NOT NULL,
        tar_cred VARCHAR(20) NOT NULL,
        c_pais SMALLINT NOT NULL, FOREIGN KEY (c_pais) REFERENCES
        pais(cod_pais) );

        ALTER TABLE cliente ADD CONSTRAINT tele_cliente UNIQUE (num_tel);
```

INSERT :

```
        INSERT INTO cliente(nombre, apellido, num_tel, email, tar_cred,
c_pais)
        SELECT DISTINCT
        SUBSTRING_INDEX(`name`,' ',1),
        SUBSTRING_INDEX(`name`,' ',-1),
        phone_number, email, credit_card, cod_pais
        FROM hotel_booking_original
        JOIN pais
        ON pais.descrip = hotel_booking_original.country;
```

NOTA: Antes de crear la tabla reserva, creamos los índices alternativos que nos ayudarán a que las consultas sean más rápidas y eficientes.

ÍNDICES ALTERNATIVOS

```
CREATE INDEX nombre_hotel
ON hotel(nombre);

CREATE INDEX descrip_comida
ON comida(descrip);

CREATE INDEX descrip_segmento
ON segmento_mercado(descrip);

CREATE INDEX descrip_canal
ON canal_distribucion(descrip);

CREATE INDEX descrip_habitacion
ON tipo_habitacion(descrip);

CREATE INDEX descrip_estado
ON estado_reserva(descrip);

CREATE INDEX descrip_cliente
ON tipo_cliente(descrip);

CREATE INDEX descrip_deposito
ON tipo_deposito(descrip);
```

RESERVA:

```
CREATE TABLE reserva(cod_reserva INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
cancelado BOOLEAN NOT NULL, dias_anticipacion SMALLINT NOT NULL,
anio_llegada SMALLINT NOT NULL, mes_llegada VARCHAR(20) NOT NULL,
semana_llegada TINYINT NOT NULL, dia_llegada TINYINT NOT NULL,
noche_finde SMALLINT NOT NULL, noche_semana SMALLINT NOT NULL,
adultos TINYINT NOT NULL,  ninios INT NOT NULL, bebes TINYINT NOT NULL,
huesped_repetido BOOLEAN NOT NULL, cancel_anteriores TINYINT NOT NULL,
no_cancel_reserva TINYINT NOT NULL,  cambio_reserva TINYINT NOT NULL,
agente VARCHAR(10), compania VARCHAR(10),  dias_lista_espera SMALLINT NOT NULL,
adr DECIMAL(6,2) NOT NULL, estacionamiento SMALLINT NOT NULL,
requisitos_esp TINYINT NOT NULL, fecha_estado_reserva DATE NOT NULL,
        rsv_cliente INT NOT NULL, FOREIGN KEY (rsv_cliente) REFERENCES
        cliente(cod_cliente),
        rsv_hotel TINYINT NOT NULL, FOREIGN KEY (rsv_hotel) REFERENCES
        hotel(cod_hotel),
        rsv_comida TINYINT NOT NULL, FOREIGN KEY (rsv_comida) REFERENCES
        comida(cod_comida),
        rsv_segme TINYINT NOT NULL, FOREIGN KEY (rsv_segme) REFERENCES
        segmento_mercado(cod_segme),
        rsv_canal TINYINT NOT NULL, FOREIGN KEY (rsv_canal) REFERENCES
        canal_distribucion(cod_canal),
        rsv_hab_r TINYINT NOT NULL, FOREIGN KEY (rsv_hab_r) REFERENCES
        tipo_habitacion(cod_habitacion),
```

```
rsv_hab_a TINYINT NOT NULL, FOREIGN KEY (rsv_hab_a) REFERENCES
tipo_habitacion(cod_habitacion),
rsv_estado TINYINT NOT NULL, FOREIGN KEY (rsv_estado) REFERENCES
estado_reserva(cod_estado),
rsv_tipocliente TINYINT NOT NULL, FOREIGN KEY (rsv_tipocliente) REFERENCES
tipo_cliente(cod_tipocliente),
rsv_deposito TINYINT NOT NULL, FOREIGN KEY (rsv_deposito) REFERENCES
tipo_deposito(cod_deposito));
```

INSERT :

```
INSERT INTO reserva( cancelado, dias_anticipacion, anio_llegada,  mes_llegada,
semana_llegada, dia_llegada, noche_finde, noche_semana, adultos, ninios, bebes,
huesped_repetido, cancel_anteriores, no_cancel_reserva, cambio_reserva, agente,
compania, dias_lista_espera, adr, estacionamiento, requisitos_esp, fecha_estado_reserva,
rsv_cliente, rsv_hotel, rsv_comida, rsv_segme, rsv_canal, rsv_hab_r, rsv_hab_a,
rsv_estado, rsv_tipocliente, rsv_deposito)
SELECT is_canceled, lead_time, arrival_date_year, arrival_date_month,
arrival_date_week_number, arrival_date_day_of_month, stays_in_weekend_nights,
stays_in_week_nights, adults, children,  babies, is_repeated_guest, previous_cancellations,
previous_bookings_not_canceled, booking_changes, agent, company, days_in_waiting_list,
adr, required_car_parking_spaces, total_of_special_requests, reservation_status_date,
c.cod_cliente, h.cod_hotel, cm.cod_comida, sm.cod_segme, cd.cod_canal,
thr.cod_habitacion, tha.cod_habitacion, er.cod_estado, tc.cod_tipocliente, td.cod_deposito
FROM hotel_booking_original hbo
    JOIN cliente c
    ON c.num_tel = hbo.phone_number
```

```
JOIN hotel h
ON h.nombre = hbo.hotel

JOIN comida cm
ON cm.descrip = hbo.meal

JOIN segmento_mercado sm
ON sm.descrip = hbo.market_segment

JOIN canal_distribucion cd
ON cd.descrip = hbo.distribution_channel

JOIN tipo_habitacion thr
ON thr.descrip = hbo.reserved_room_type

JOIN tipo_habitacion tha
ON tha.descrip = hbo.assigned_room_type

JOIN estado_reserva er
ON er.descrip = hbo.reservation_status

JOIN tipo_cliente tc
ON tc.descrip = hbo.customer_type

JOIN tipo_deposito td
ON td.descrip = hbo.deposit_type;
```

VISTAS

VISTA 1: Muestra cuántas reservas canceladas tiene cada hotel.

```
CREATE VIEW vista_cancelaciones_hotel AS
SELECT
    h.nombre AS hotel,
    COUNT(*) AS total_canceladas
FROM reserva r
JOIN hotel h
    ON r.rsv_hotel = h.cod_hotel
WHERE r.cancelado = 1
GROUP BY h.cod_hotel, h.nombre;
```

VISTA 2: Muestra las reservas de 5 o más noches, sumando las de finde y las de semana

```
CREATE VIEW vista_estadias_largas AS
SELECT
    cod_reserva, mes_llegada, anio_llegada,
    noche_finde, noche_semana,
    noche_finde + noche_semana AS total_noches
FROM reserva
WHERE noche_finde + noche_semana >= 5;
```

VISTA 3: Que mes se registraron más reservas

```
CREATE VIEW vista_reservas_por_mes AS
        SELECT anio_llegada, mes_llegada,
        COUNT(*) AS cantidad_reservas FROM reserva
GROUP BY anio_llegada, mes_llegada;
```

VISTA 4: Muestra la reserva junto con el hotel, el estado,el tipo de habitación asignada y el importe

```
CREATE VIEW vista_reservas AS
        SELECT r.cod_reserva, h.nombre AS hotel, r.mes_llegada,
        r.anio_llegada, r.adultos, r.ninios, er.descrip AS estado_reserva,
        tha.descrip AS habitacion_asignada, r.adr
FROM reserva r
        JOIN hotel h
        ON r.rsv_hotel = h.cod_hotel
        JOIN estado_reserva er
        ON r.rsv_estado = er.cod_estado
        JOIN tipo_habitacion tha
        ON r.rsv_hab_a = tha.cod_habitacion;
```

ADR mayores a >100

VISTA 5: Filtra los ADR superiores a 100

```
CREATE VIEW vista_reservas_adr_alto AS
SELECT cod_reserva, adr, adultos, ninios, mes_llegada, anio_llegada
FROM reserva WHERE adr >100;
```

CONSULTAS MÁS FRECUENTES

CONTAR REGISTROS:
SELECT COUNT(*) FROM hotel_booking;

VER DATOS:

SELECT * FROM hotel_booking;

OBTENER VALORES NO REPETIDOS:

SELECT DISTINCT columna

FROM hotel_booking;

CONTAR Y AGRUPAR DATOS

SELECT columna, COUNT(*)

FROM hotel_booking

GROUP BY columna;

CONSULTAR TABLAS NORMALIZADAS:

SELECT * FROM nombre_tabla;